

Installation d'un poste pour le développement

SAÉ S1.03

Installation d'un poste pour le développement



UNIVERSITÉ
DE LORRAINE



IUT Saint-Dié-des-Vosges

Établissement/Formation

IUT de Saint-Dié-Des-Vosges - Première Année
BUT Informatique

Réalisé par:

Antoine Barthélémy, Julien Durney, Milo Bastien

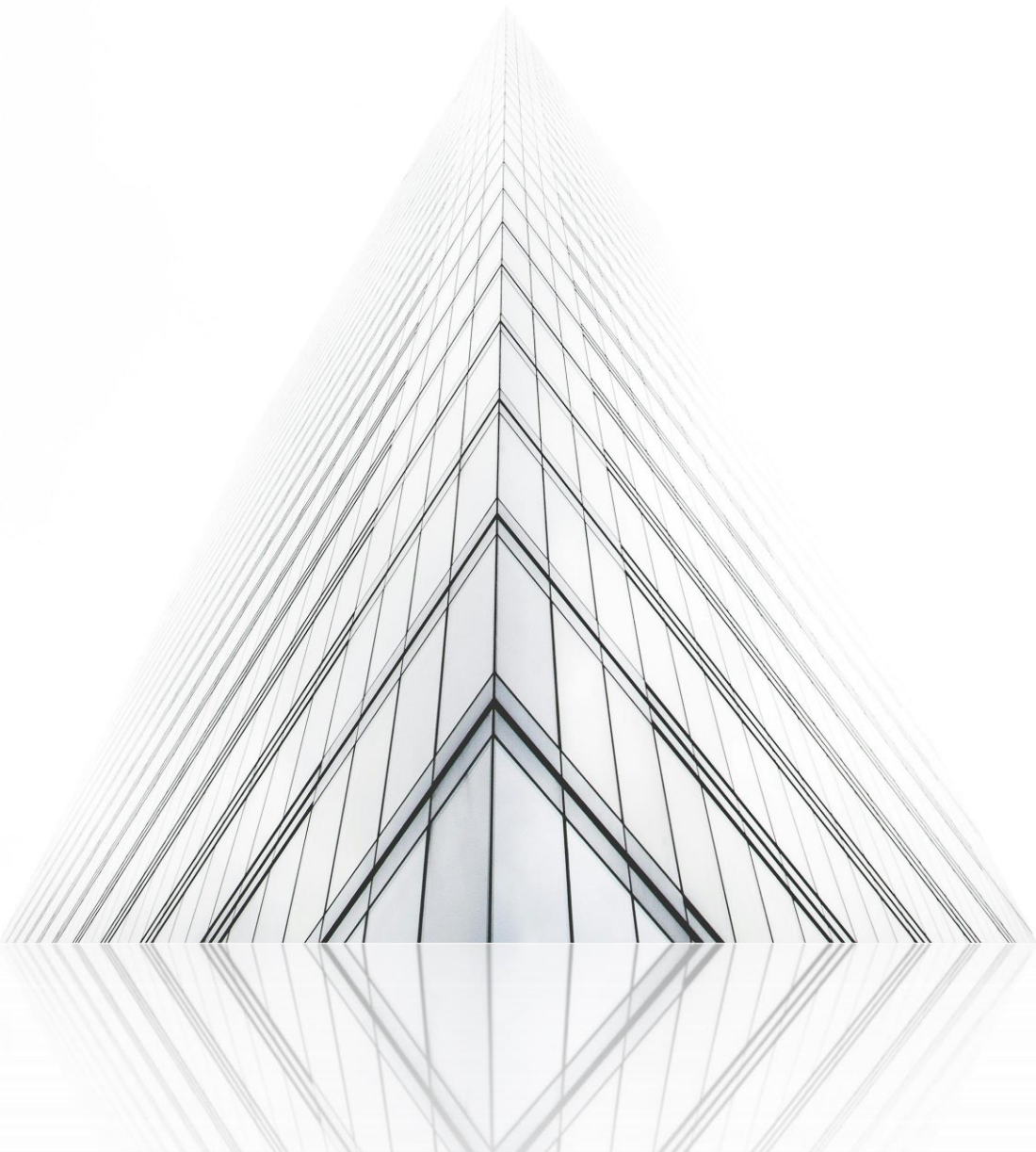
Encadrant:

Abdellatif Bourjij

Sommaire

Contents

Introduction.....	3
Identification de notre collaborateur et de ses missions :.....	3
Comparaison des systèmes d'exploitation présents sur le marché :	4
Choix du système d'exploitation :	9
Guide d'installation :	9
Schéma de l'architecture logicielle :	18



Introduction

Dans le cadre de notre formation en BUT1 Informatique à l'IUT de Saint-Dié-des-Vosges, nous avons été chargés de réaliser l'installation d'un système d'exploitation adapté au développement. Cette activité s'inscrit dans la SAÉ de l'unité d'enseignement 1.3, intitulée « Gérer », et vise à développer nos compétences en configuration et gestion de systèmes informatiques.

Ce rapport présente une analyse détaillée de cette installation, en suivant une approche structurée pour permettre une compréhension progressive des étapes et des choix réalisés. Vous y découvrirez le contexte, les outils employés, les défis rencontrés, ainsi que les résultats obtenus.

Afin de garantir une lecture efficace, nous recommandons de suivre ce rapport dans l'ordre proposé. La structure sera expliquée en détail dans la section suivante, « Définition de la structure de notre rapport ».

Nous nous plaçons dans le contexte fictif d'une entreprise qui se prépare à accueillir un nouveau salarié. Ce salarié, un développeur, utilisera dans le cadre de son métier les outils que nous avons installés et configurés. Comme ces informations sont fictives, nous avons défini notre collaborateur dans la section « Identification de notre collaborateur et ses missions ».

Définition de la structure de notre rapport :

- Identification de notre collaborateur et de ses missions
- Comparaison des systèmes d'exploitation présents sur le marché
- Choix du système d'exploitation
- Guide d'installation
- Schéma de l'architecture logicielle

Identification de notre collaborateur et de ses missions :

Raphael est un ingénieur spécialisé dans le développement d'applications web, utilisant divers langages de programmation dans l'IDE Visual Studio Code. Ses missions au sein de notre entreprise consisteront principalement à réaliser la refonte de notre site internet, incluant le front-end, le back-end, et le déploiement.

Le déploiement de notre site nécessitera des compétences variées, notamment la maîtrise des commandes Bash. En effet, nous n'utilisons pas d'interfaces graphiques pour communiquer avec notre serveur web.

Comparaison des systèmes d'exploitation présents sur le marché :

Linux :

Buts et usages à sa création

- **Naissance en 1991** : Créé par Linus Torvalds, Linux est conçu comme un système d'exploitation libre inspiré d'Unix. Initialement, il s'agissait d'un projet personnel pour créer un noyau open source utilisable par tous.
- **Philosophie** : Fondé sur les principes du logiciel libre, Linux incarne la liberté de modifier, distribuer et utiliser le logiciel sans restrictions majeures, suivant la licence GPL de Richard Stallman.

Évolutions au fil du temps

- **Adoption massive dans les serveurs** : Dès les années 2000, Linux s'est imposé dans l'infrastructure des serveurs grâce à sa fiabilité et à son coût réduit par rapport à des systèmes propriétaires.
- **Diversité des distributions** : La modularité a conduit à des distributions adaptées à des usages variés, des débutants (Ubuntu) aux experts (Arch Linux, Debian).
- **Élargissement de son influence** : Il alimente aujourd'hui des systèmes variés comme Android pour smartphones, des objets connectés, et des supercalculateurs.

Objectifs actuels

- **Innovation continue** : Rester un système évolutif pour des besoins variés, allant du grand public aux infrastructures critiques.
- **Sécurité et performances** : Maintenir sa stabilité et combler les failles rapidement grâce à une vaste communauté mondiale.

Caractéristiques clés

- **Nature open source** : Tout le code source est librement accessible et modifiable.
- **Personnalisation extrême** : Une multitude de distributions permet d'adapter Linux à des besoins spécifiques.
- **Fiabilité** : Très peu de plantages, idéal pour des environnements exigeants.
- **Communauté dynamique** : Une amélioration collaborative et des mises à jour régulières.
- **Interopérabilité** : Compatible avec de nombreux standards et formats ouverts, il favorise l'intégration avec divers systèmes.

Windows :

Buts et usages à sa création

- **Naissance en 1985** : La première version de Windows, Windows 1.0, a été lancée le 20 novembre 1985 en tant qu'interface graphique pour MS-DOS. Son objectif initial était de simplifier l'utilisation des ordinateurs en rendant l'informatique accessible à un plus large public.
- **Philosophie** : Windows visait à remplacer l'interface en ligne de commande de MS-DOS par une interface utilisateur graphique (GUI) conviviale, utilisant des fenêtres, des icônes, et des menus. Cela a permis aux utilisateurs de gagner en productivité et de bénéficier d'un environnement multitâche.

Évolutions au fil du temps

- **Adoption massive dans les postes de travail** : Dès les années 1990, Windows s'est imposé comme le système d'exploitation standard pour les ordinateurs personnels, avec une compatibilité étendue pour les logiciels et les périphériques.
- **Diversification des offres** : Microsoft a introduit des versions adaptées aux entreprises (Windows NT), aux serveurs, et même aux systèmes embarqués comme les distributeurs automatiques de billets.
- **Intégration au cloud** : Plus récemment, avec Windows 365 et le support de Windows Subsystem for Linux (WSL), Windows a évolué pour répondre aux besoins modernes des entreprises et des développeurs.

Objectifs actuels

- **Accessibilité universelle** : Fournir un environnement simple et intuitif adapté aussi bien aux particuliers qu'aux entreprises.
- **Environnement de développement complet** : Offrir des outils intégrés pour les développeurs, comme WSL, afin de travailler avec des technologies Linux au sein de Windows.
- **Productivité et connectivité** : Intégration avec des outils comme Microsoft Office, OneDrive, et Teams pour favoriser la collaboration et le travail en ligne.

Caractéristiques clés

- **Compatibilité** : Windows prend en charge une grande variété de matériels et de logiciels, facilitant l'utilisation sur de nombreux périphériques.
- **Interface utilisateur** : Conçue pour être simple et intuitive, elle ne nécessite pas de connaissances préalables pour être utilisée efficacement.
- **Personnalisation** : Bien que limitée comparée à Linux, Windows propose des options pour personnaliser l'apparence et les paramètres système.
- **Sécurité** : Intégration de fonctionnalités comme Windows Defender, bien que son modèle propriétaire limite l'inspection du code source par des tiers.
- **Coût** : Windows est payant, avec des versions accessibles comme Windows 10 Home, proposées à des prix réduits.

Mac OS :

Buts et usages à sa création

- **Successeur d'anciennes versions Mac OS** : Développé par Apple, macOS a vu le jour en 2001 en succédant à Mac OS Classic. Il intègre des technologies issues de NeXT, une société acquise par Apple en 1997.
- **Vision intégrée** : Fournir un système conçu pour tirer parti de l'écosystème Apple et offrir une interface utilisateur intuitive et performante.

Évolutions au fil du temps

- **Modernisation continue** : À partir de 2001, Apple a régulièrement ajouté des fonctionnalités modernes comme Spotlight (recherche), Time Machine (sauvegarde), et une intégration croissante avec iOS.
- **Optimisation matérielle** : Avec l'introduction des processeurs Apple Silicon, macOS est devenu encore plus fluide et performant grâce à une synergie matérielle-logicielle.

Objectifs actuels

- **Convergence de l'écosystème Apple** : Maximiser l'interconnexion avec d'autres produits Apple (iPhone, iPad) pour une expérience utilisateur unifiée.
- **Confidentialité et sécurité** : Met l'accent sur la protection des données personnelles avec des fonctionnalités comme la gestion stricte des permissions.

Caractéristiques clés

- **Interface utilisateur avancée** : Design attrayant et cohérent grâce à l'environnement Aqua.
- **Écosystème exclusif** : Synchronisation fluide avec iCloud et des applications Apple comme GarageBand et iMovie.
- **Optimisation** : Exploite pleinement le matériel propriétaire pour des performances exceptionnelles.
- **Sécurité renforcée** : macOS bénéficie d'une architecture Unix solide avec des mises à jour régulières et un focus sur la confidentialité.

Avantages et désavantage de Windows, Mac OS et du noyau Linux :

OS	Avantages	Désavantage
Windows	• Compatibilité importante (hardware, software). • Facile à utiliser et à apprendre. • Windows offre un bon support pour les logiciels propriétaires et les jeux. • Mise à jour régulières, ce qui aide à améliorer la sécurité et la stabilité du système. • Sécurité accrue grâce à Windows Defender • Windows propose plusieurs éditions (Home, Pro, Entreprise) adaptées à différents besoins.	• Windows est payant ce qui peut freiner certains utilisateurs. • Les mises à jour régulières provoquent des temps où le système est inutilisable. • Le système est moins personnalisable ce qui limite les actions possibles sur la machine. • Plus à même de recevoir des attaques dû à une plus grande base d'utilisateurs. • Exigeant en termes de ressources mémoire. • Collecte d'informations personnelles importantes.
Linux	• Linux est entièrement gratuit et son code source est accessible, permettant une personnalisation totale et une adaptation à divers besoins spécifiques. • Sécurité renforcée : Grâce à son architecture et à sa communauté active, Linux est très résistant aux logiciels malveillants et bénéficie de mises à jour régulières. • Performance et stabilité : Linux est un choix privilégié pour les serveurs web en raison de sa capacité à gérer des charges de travail importantes tout en maintenant une performance stable sur le long terme. Cette caractéristique est particulièrement	• Moins intuitif pour les débutants, nécessitant parfois l'usage de la ligne de commande. • Compatibilité logicielle limitée : Certains logiciels et jeux propriétaires ne sont pas disponibles sur Linux. • Quelques périphériques récents peuvent ne pas être pleinement compatibles. • Support professionnel : Bien que la communauté soit active, le support professionnel est moins accessible

	avantageuse pour les développeurs, car elle permet de reproduire les conditions réelles dans lesquelles une page web sera hébergée. • Polyvalence et personnalisation : De nombreuses distributions (Ubuntu, Fedora, etc.) permettent de choisir l'interface et les fonctionnalités adaptées. • Outils de développement : Linux possède une vaste gamme d'outils et de logiciels préinstallés et accessibles aux développeurs. • Compatibilité matérielle large : Linux peut fonctionner sur des ordinateurs anciens ou peu puissants, contrairement à d'autres systèmes qui nécessitent du matériel récent.	qu'avec des systèmes comme Windows ou Mac OS.
MacOS	• Interface réputée pour son design épuré et sa facilité d'utilisation. • Écosystème intégré : Une synchronisation optimale avec les appareils Apple (iPhone, iPad, etc.), facilitant les échanges de données et la continuité d'usage. • Les systèmes Mac OS sont conçus pour maximiser les capacités matérielles des appareils Apple, garantissant fluidité et rapidité. • Mac OS offre un environnement relativement sécurisé grâce à des mises à jour régulières et une architecture robuste.	• Coût élevé : Les appareils nécessaires pour utiliser Mac OS sont coûteux, avec peu d'options pour réduire ces frais. • Flexibilité limitée : Contrairement à Linux ou Windows, Mac OS est propriétaire et offre peu de possibilités de personnalisation. • Compatibilité limitée en dehors de l'écosystème Apple : Les utilisateurs peuvent rencontrer des difficultés avec des logiciels ou périphériques non optimisés pour Mac OS.

Choix du système d'exploitation :

Pour Raphael, nous avons choisi Linux. En effet, ce système (noyau) correspond le mieux à son profil. Il offre une grande sécurité, des outils de développement intégrés, et permet d'effectuer des tâches complexes. Qui plus est, une majorité des serveurs web fonctionnent avec une distribution Linux, ce qui apporte un avantage considérable et permet de travailler dans des conditions similaires à celles des environnements de production.

Nous n'avons pas retenu Windows, car nous souhaitons que Raphael travaille dans des conditions proches de celles de production. Quant à Mac OS, nous l'avons écarté en raison de ses coûts élevés, qui le rendent inadapté à une entreprise comme la nôtre.

Guide d'installation :

Introduction

Dans ce guide, vous trouverez toutes les informations relatives à l'installation de la distribution Linux. Avant de poursuivre votre lecture, il est important de clarifier le vocabulaire qui sera employé. Linux n'est pas un système d'exploitation, mais la première couche, celle qui interagit directement avec les composants de l'ordinateur. Les systèmes d'exploitation basés sur ce noyau sont appelés des « distributions ». Lorsque l'on dit que Linux est Open Source, cela signifie que tout le monde peut accéder librement au code du noyau et ainsi construire sa propre distribution. Le BIOS est un logiciel qui agit comme un pont entre les composants d'un ordinateur et le système d'exploitation, le BIOS a été remplacé par l'UEFI. Je vous laisse vous renseigner sur ces sujets si cela vous intéressent.

Pour procéder à l'installation de ce dernier, il faut suivre quatre étapes, lesquelles sont répertoriées à l'adresse suivante : <https://learn.microsoft.com/en-us/linux/install>. Pour simplifier la lecture, je les partage ici :

1. Choisir la méthode pour installer Linux
2. Choisir la distribution Linux
3. Suivre les méthodes d'installation
4. Après l'installation de Linux

Je ne peux pas répertorier toutes les méthodes d'installation de Linux ni toutes les distributions existantes, car cela prendrait énormément de temps et ce n'est pas l'objectif de cette SAÉ. Cependant, je vous expliquerai en détail mes choix ainsi que les réflexions à avoir lorsque l'on souhaite installer un système d'exploitation sur sa machine. Le guide d'installation sera donc construit autour de la ressource présenté à l'url donné.

Choisir la méthode pour installer Linux :

Le choix de la méthode pour installer Linux dépend de vos besoins et préférences. Microsoft répertorie trois catégories principales :

1. Windows Subsystem for Linux (WSL)

- **Pour qui ?** Les utilisateurs débutants ou ceux qui souhaitent découvrir Linux de manière simple et rapide.
- **Avantages :** Cette méthode permet d'utiliser Linux directement depuis Windows, sans avoir à créer une machine virtuelle ou modifier le système d'exploitation existant. Elle est idéale pour tester Linux tout en conservant l'accès à vos outils Windows, comme Outlook, Teams ou Microsoft Office.

2. Machine virtuelle (VM) dans le cloud ou en local

- **Pour qui ?** Les professionnels travaillant dans des environnements complexes nécessitant une échelle ou une sécurité accrue, ou pour ceux qui souhaitent exécuter Linux comme serveur.
- **Avantages :** Exécuter Linux dans une machine virtuelle permet de l'isoler totalement du système principal. Microsoft recommande de considérer des solutions cloud, comme Azure, qui offrent un support robuste pour les configurations à grande échelle.

3. Linux en tant que système d'exploitation principal (Bare Metal)

- **Pour qui ?** Les utilisateurs avancés qui souhaitent utiliser Linux comme leur système d'exploitation principal et tirer parti de tout le potentiel de leur matériel.
- **Avantages :** Cette méthode élimine toute virtualisation ou émulation, offrant ainsi des performances maximales. Cependant, elle nécessite une installation plus complexe et ne donne pas accès aux outils Windows tels qu'Outlook ou Teams.

Conclusion

Pour cette première étape, j'ai décidé d'opter pour la 3^{ème} méthode d'installation. En effet, je souhaitais bénéficier de l'avantage d'avoir deux systèmes d'exploitation sur ma machine et, par conséquent, pouvoir choisir au démarrage quel système d'exploitation utiliser. De plus, n'ayant jamais utilisé cette méthode d'installation auparavant, je souhaitais acquérir de nouvelles compétences.

Choisir la distribution Linux

Il est essentiel de choisir une distribution Linux qui corresponde aux besoins de Raphael. Cette étape est l'une des plus importantes et décisives lorsque l'on souhaite installer Linux. De nombreux facteurs doivent être pris en compte pour faire ce choix. Microsoft identifie quatre critères clés :

1. Niveau d'expérience avec Linux
2. Exigences système
3. Sécurité
4. Environnements professionnels

Conclusion

Nous avons opté pour POP!_OS, un système d'exploitation puissant, léger et orienté vers les développeurs. Il correspond parfaitement à nos objectifs : offrir rapidité et efficacité pour répondre aux besoins de Raphael. Ce système fournit un environnement entièrement conçu pour les développeurs, avec des logiciels préinstallés comme Git, Docker et Python, ainsi que des compilateurs intégrés. En outre, son implémentation garantit une utilisation agréable tout en favorisant la stabilité. POP!_OS permet également d'optimiser les flux de travail grâce à l'organisation automatique des fenêtres, ce qui fait gagner un temps précieux lors du multitâche entre le développement front-end et back-end.

Suivre les méthodes d'installation :

La méthode d'installation dépend de celle que vous avez choisie pour installer Linux. Pour la méthode que j'ai sélectionnée, « Bare Metal », Microsoft répertorie quatre grands axes à suivre dans un ordre précis. Ces étapes seront illustrées par des images, afin de vous permettre de suivre de près mon installation de POP!_OS et, pourquoi pas, de l'installer vous-même. Voici ces quatre étapes :

1. Télécharger le fichier image de la distribution :

- Commencez par télécharger un fichier image (généralement un fichier ISO) correspondant à la version de votre distribution Linux. Assurez-vous de consulter le site officiel de la distribution choisie pour trouver la version la plus récente.
- Certains fichiers ISO nécessitent une vérification de signature avant installation. De plus, certaines distributions requièrent la désactivation de "Secure Boot" sur Windows, ce qui n'est généralement pas recommandé.

2. Créer une clé USB bootable :

- Vous aurez besoin d'une clé USB d'au moins 16 Go et d'un logiciel pour créer la clé bootable (par exemple, balenaEtcher, Rufus ou UNetbootin). La plupart des sites officiels de distributions Linux recommandent un outil spécifique pour cette tâche.

3. Démarrer votre appareil depuis la clé USB :

- Redémarrez votre appareil et accédez au menu de démarrage (généralement en appuyant sur la touche **F12** lors du démarrage). Sélectionnez ensuite votre clé USB comme périphérique de démarrage pour lancer l'installation de Linux.

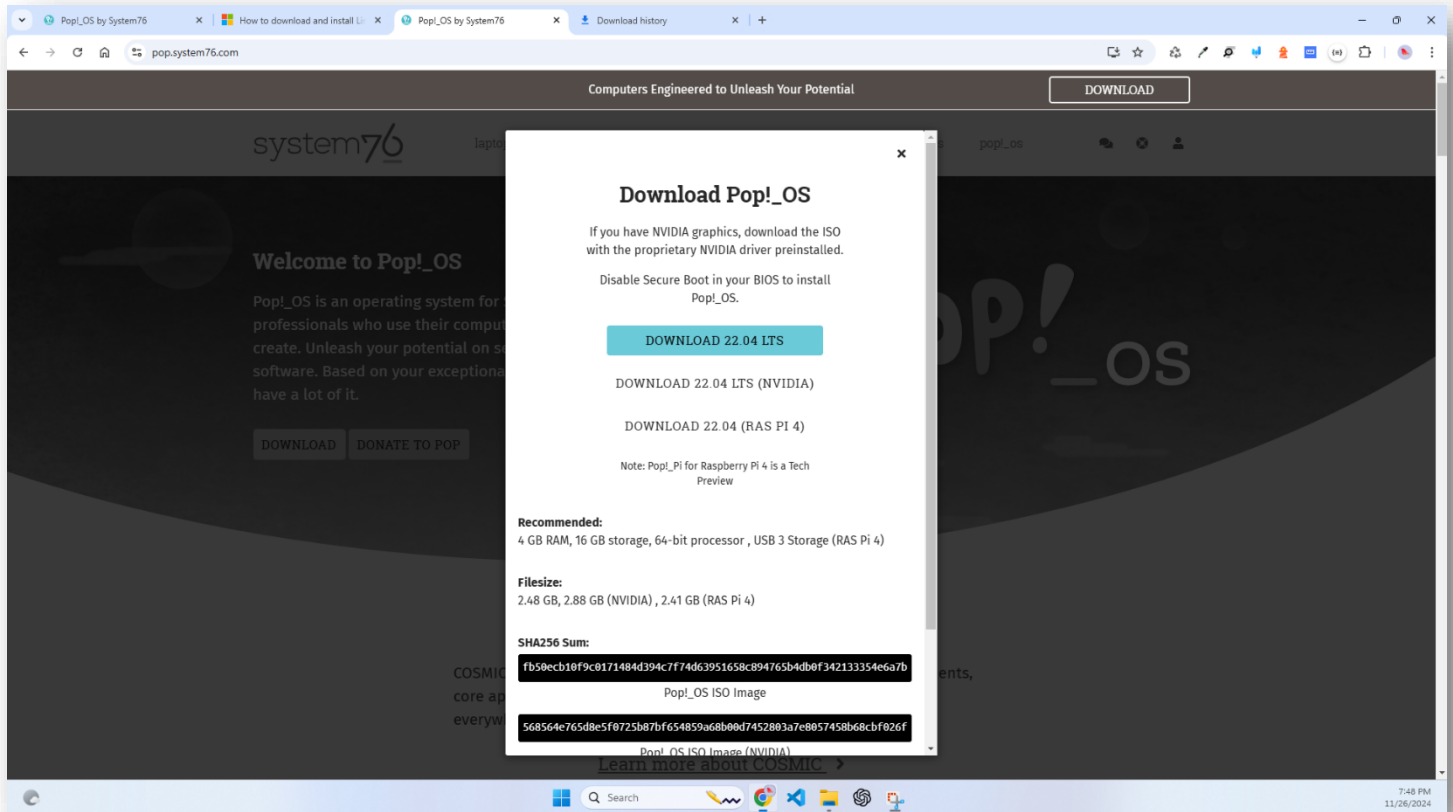
4. Configurer l'installation :

- Suivez les étapes de l'installateur de votre distribution. Ces étapes incluront des choix tels que :
 - Inclure ou non certains logiciels tiers.
 - Utiliser le chiffrement pour sécuriser vos données.
 - Partitionner le disque si vous souhaitez conserver Windows en mode dual-boot, ou effacer complètement le disque pour installer uniquement Linux.
- À la fin du processus, vous serez invité à créer un nom d'utilisateur et un mot de passe.

1. Télécharger le fichier image de la distribution

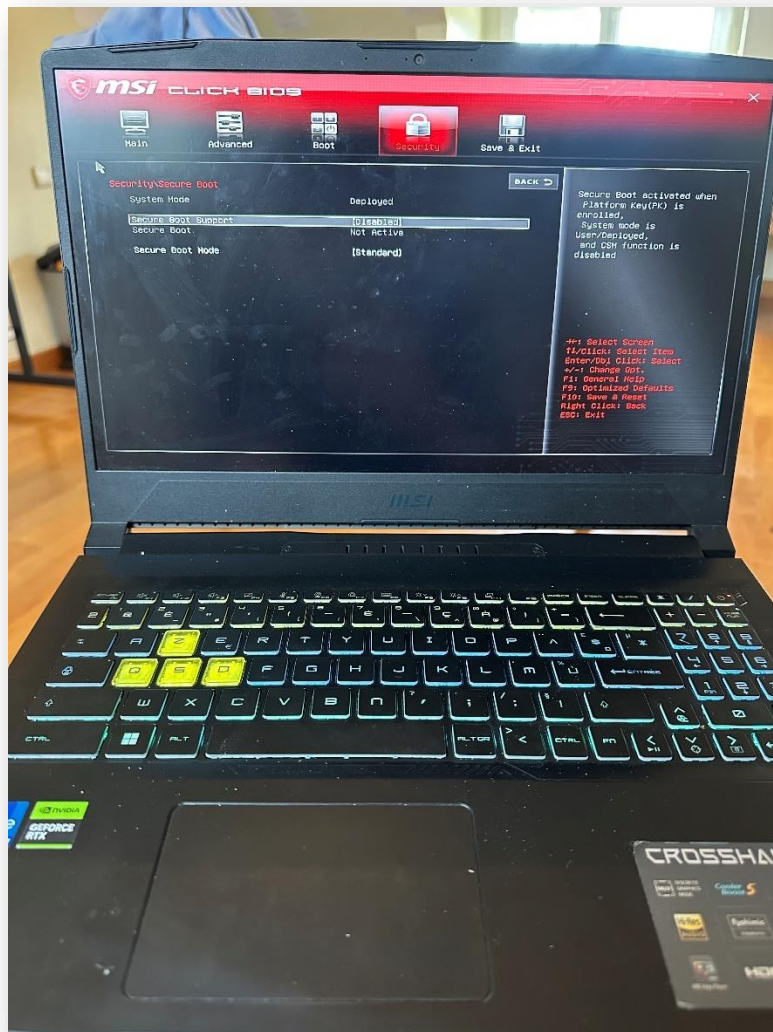
En suivant la méthodologie, nous nous sommes retrouvés sur le site internet de System76 (l'entreprise qui a créé POP!_OS). En parcourant le site, nous avons finalement trouvé l'ISO qu'il nous fallait. Celle-ci est disponible à l'adresse suivante : <https://pop.system76.com/>.

Voici un aperçu de ce processus :



Il est indiqué qu'il faut désactiver le « Secure Boot », autrement dit le démarrage sécurisé, pour procéder à l'installation de POP!_OS. Cependant, à cette étape, nous installons simplement l'image du système d'exploitation, ce qui ne constitue pas un obstacle pour le moment. La désactivation du Secure Boot sera effectuée juste après. Pour désactiver le secure boot, il faut se rendre dans le [BIOS/UEFI](#)

Voici une image illustrant l'interface du BIOS/UEFI et l'endroit où figure cette option :



Créer une clé USB bootable :

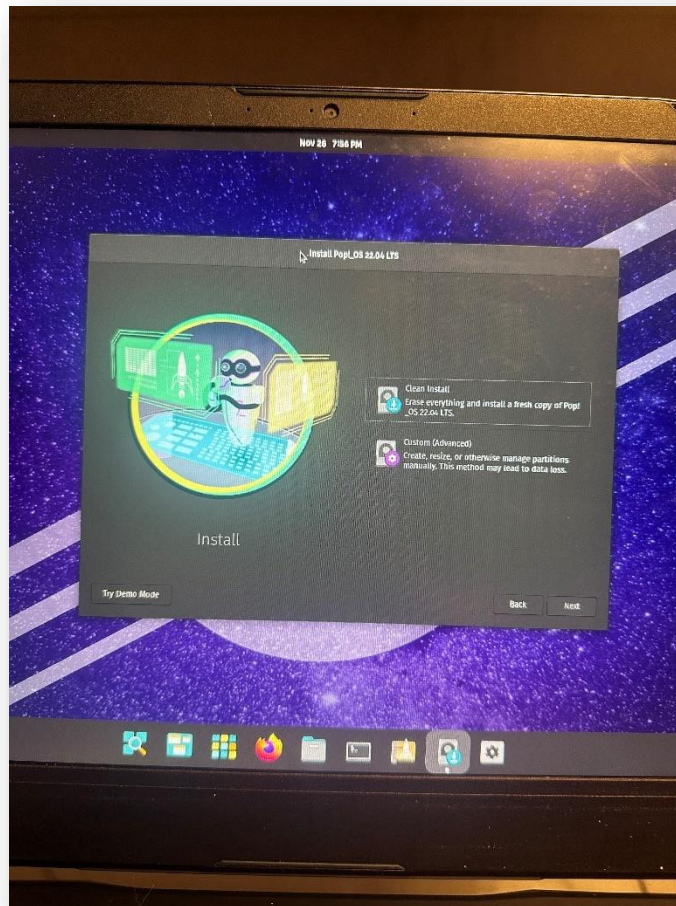
Transformer notre clé USB en clé USB bootable est une étape cruciale, car sans cela, nous ne pourrions pas démarrer depuis la clé USB. En choisissant la méthode d'installation 3 (« Bare Metal »), nous savions que cette étape était incontournable. En effet, elle permet à l'ordinateur d'exécuter le système d'exploitation depuis la clé USB. De plus, rendre une clé USB bootable permet d'y installer le « [bootloader](#) », un programme essentiel au démarrage d'un système d'exploitation sur une machine. Pour cette étape, j'ai choisi d'utiliser [Rufus](#) (un logiciel permettant de rendre une clé USB bootable), car je l'avais déjà utilisé dans le passé et je sais qu'il fonctionne très bien.

Voici une image illustrant le processus pour rendre une clé USB bootable grâce à Rufus :

Si vous réussissez à voir votre clé USB dans le menu de démarrage, cela signifie que votre clé a été correctement convertie en support bootable par votre système.

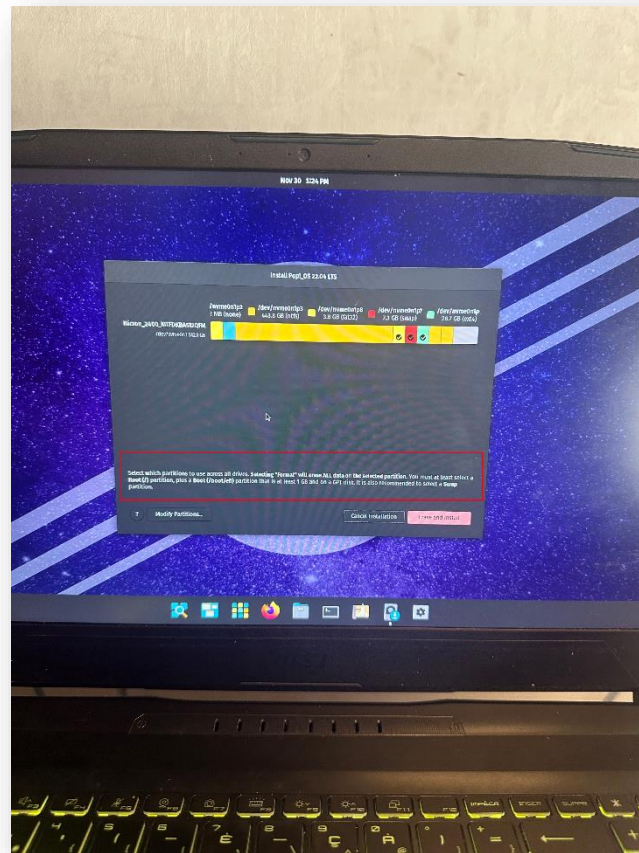
Configurer l'installation :

Lorsque l'on réussit à accéder à la page d'accueil de notre système d'exploitation, il est important de comprendre que ce système n'a pas encore été installé sur notre SSD. Notre machine a « simplement » réussi à démarrer correctement le système d'exploitation depuis la clé USB. Quelques réglages essentiels et universels sont demandés à l'utilisateur, comme la langue du système, la disposition du clavier, etc., jusqu'à un point important de configuration qu'il faut souligner.



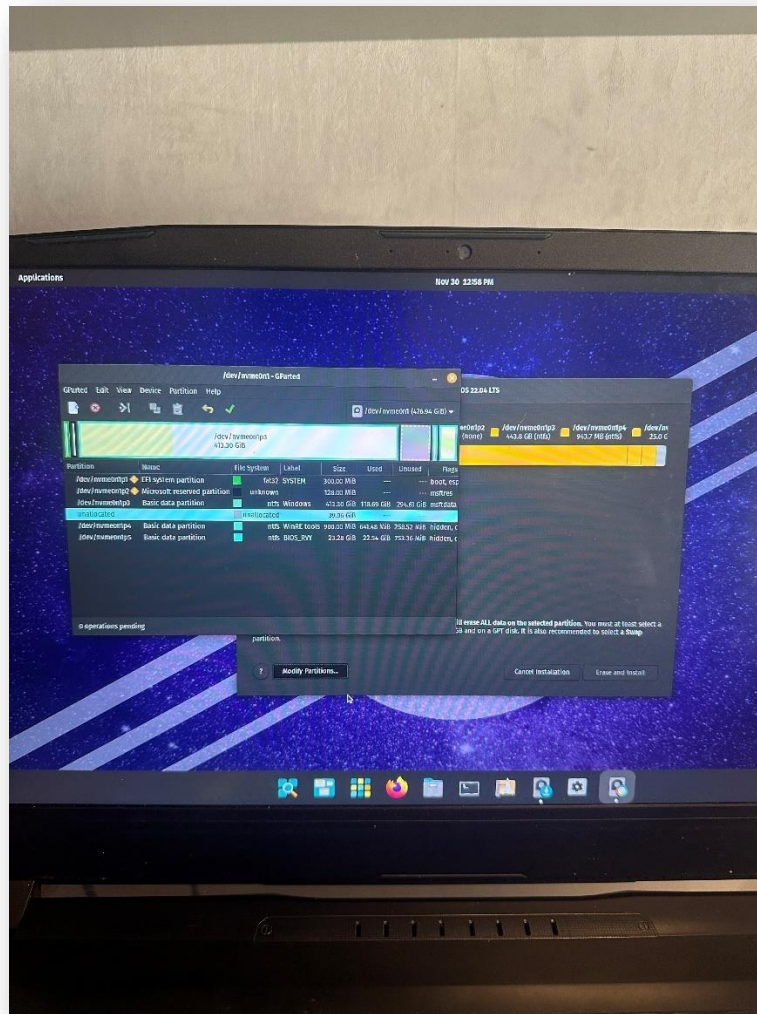
La première option supprime absolument tout sur le disque, y compris Windows, et ne laisse que Pop!_OS comme seul système d'exploitation. Comme mentionné plus haut dans la section “Choisir une méthode d'installation pour Linux”, je souhaite pouvoir bénéficier de deux systèmes d'exploitation. Par conséquent, j'ai choisi la deuxième option, qui permet de partitionner le disque manuellement et ainsi de préserver Windows.

Une fois la deuxième option sélectionnée, nous arrivons sur l'image ci-dessous. Il est indiqué qu'il faut créer au minimum trois partitions : la partition Root, la partition Boot, et la partition Swap.



POP!_OS nous propose, ce qui n'est pas toujours le cas, un gestionnaire de disques. Ce gestionnaire permet d'effectuer des opérations avancées de stockage, comme le choix du système de fichiers, la taille des partitions, et bien plus.

Les systèmes de fichiers (comme NTFS, FAT32 ou EXT4) définissent la manière dont les fichiers sont stockés et récupérés sur le disque. Une bonne analogie serait de comparer un disque à une bibliothèque : les étagères représentent l'endroit où les livres (fichiers) sont stockés, et la bibliothécaire (le système de fichiers) connaît l'emplacement de chaque livre dans la bibliothèque. Elle gère les livres entrants et sortants, et s'assure qu'aucun livre ne disparaît. Je ne vais pas rentrer dans les détails ici, cependant, je vous invite à vous renseigner davantage sur ce sujet. Il me semblait important de vous expliquer ces informations, car elle apparaissent fréquemment lorsque l'on manipule le stockage.



Si vos partitions correspondent à ce qui est attendu, vous serez invité à installer le système d’exploitation sur votre machine. Ce dernier occupera les trois partitions créées, chacune ayant un rôle bien précis, brièvement expliqué ci-dessous :

1. La première partition, appelée “Root” :

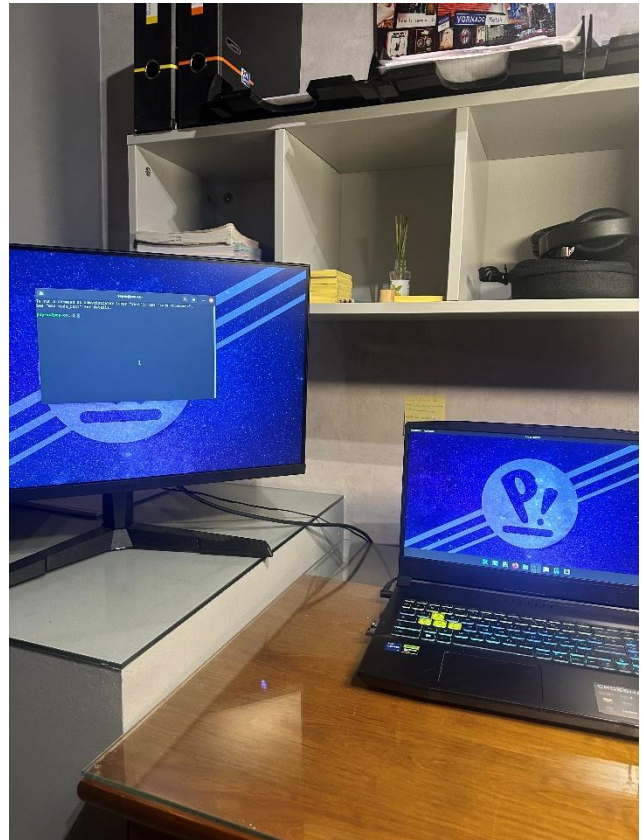
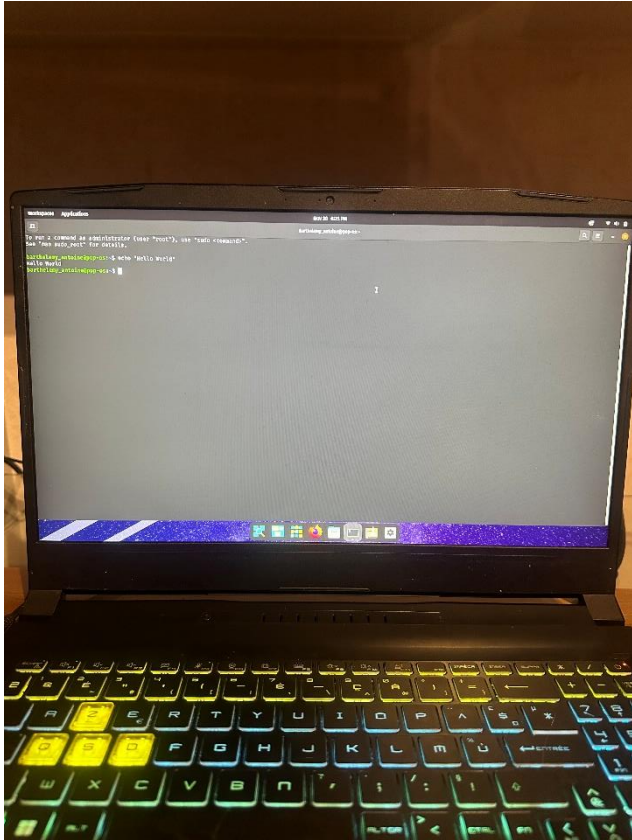
Elle agit comme le maître d’orchestre. Elle contient tous les fichiers systèmes et de configuration. Il s’agit du fameux “root directory” (/) qui représente, dans l’arborescence, le point le plus haut.

2. La deuxième partition, appelée “Swap partition” :

Elle permet de reproduire le comportement de la RAM lorsque celle-ci est surchargée, en offrant une extension temporaire de mémoire.

3. La troisième partition, appelée “Boot Partition” :

Elle contient les instructions nécessaires pour indiquer à l’ordinateur comment démarrer le système d’exploitation.



Conclusion

Nous voici arrivés au terme de ce guide d'installation. Si vous avez suivi cette installation de votre côté, une bonne pratique consiste à tester la réactivité de votre système avec un petit « Hello, World ! ». J'espère que vous avez appris de nombreux termes techniques qui vous seront utiles. Comme je l'ai mentionné, je vous encourage à approfondir vos connaissances de votre côté : ce domaine regorge d'opportunités et d'éléments intéressants à découvrir.

Schéma de l'architecture logicielle :

Introduction

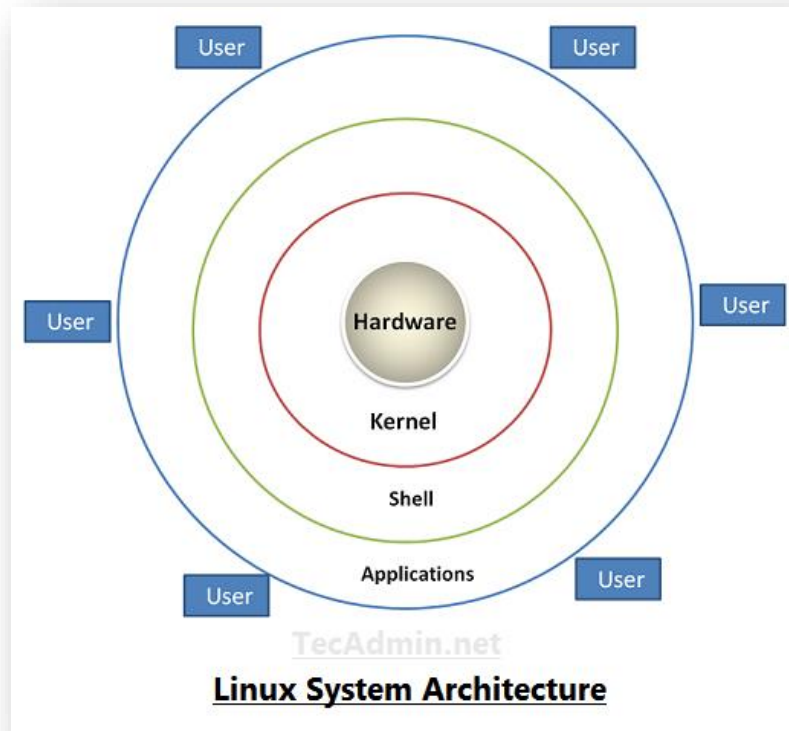
Dans la dernière partie de ce rapport, il nous a été demandé de décrire l'architecture logicielle. Naturellement, on peut se demander : « Qu'est-ce que l'architecture logicielle ? ». L'architecture logicielle, c'est l'ensemble du cheminement incluant le choix des logiciels, leur installation, leur configuration et, enfin, la manière dont ces logiciels collaborent ensemble pour créer un environnement fonctionnel pour Raphaël.

Le choix des logiciels et leur configuration dépendront donc des besoins de Raphaël. Dans le cadre de notre formation, il nous a été demandé d'installer les logiciels « les plus utilisés » dans les semestres S1 et S2. Plus spécifiquement, comme Raphaël est un développeur web, nous nous attarderons sur les logiciels utilisés et installés dans les ressources propres au développement web. En nous concentrant sur la partie logiciel, nous nous

intéressons en réalité à la couche 3 de l'architecture Linux, la couche « Kernel » et « Shell » étant principalement couvert lors de l'installation du système d'exploitation.

Architecture de Linux

Schéma illustrant l'Architecture Linux :



Il me semblait important de vous préciser le contexte dans lequel on travaille avant d'aborder l'architecture logicielle, c'est pour cela que nous avons décidé de vous parler brièvement des rôles et implications de chaque couche exprimé ci-dessous.

- **Hardware (Composants d'un ordinateur)**

Ram, CPU, Carte graphique ... Ces composants sont déterminés et choisis en fonction de l'usage qu'aura l'utilisateur avec sa machine, il est le point d'arrivée de l'information.

- **Kernel (Noyau)**

Le kernel assure la communication entre les applications et le matériel, c'est le cœur du système d'exploitation. Le kernel est installé avec l'OS (Nous ne couvrirons pas cette couche dans cette partie.)

- **Shell**

Le shell est l'interface qui permet aux utilisateurs et aux applications d'interagir avec le kernel, par exemple une instruction comme « ls » est faite dans le shell. A la suite d'une instruction comme « ls » le

shell demande au kernel de charger et d'exécuter le fichier binaire associé à « ls ». Le kernel agit donc comme un pont reliant deux rive, la partie gauche dite « de bas niveaux » et la partie droite dite « de haut niveaux ». Cette architecture est présent dans la majorité des systèmes d'aujourd'hui et dans de nombreux domaines. Il y'a toujours une traduction entre le cœur d'un système et l'action de la personne utilisant ce système.

- **Applications**

Cette couche contient en son sein, l'architecture logicielle. Mais en fait, il s'agit ni-plus ni moins d'une surcouche facilitant l'utilisation de son ordinateur, cette surcouche est plus ou moins épaisse en fonction du système d'exploitation choisie. C'est depuis cette couche, que sont lancée la plupart des demandes, car elle favorise la productivité.

- **User (l'utilisateur)**

L'utilisateur représente dans notre cas Raphael, il exprime une demande matérialisé par une action, cette action est traduite en passant de la partie droite de la rive à la partie gauche.

Applications

Après avoir étudié et compris l'architecture Linux, il est plus facile de comprendre et de réaliser une architecture logicielle réussie. Pour cela, nous devons définir les logiciels à installer. En plus de Visual Studio Code (IDE), nous installerons Apache, MySQL, PHP et phpMyAdmin. Pourquoi ces choix ? Voici les explications :

- **Visual Studio Code**

Visual Studio Code est un environnement de développement intégré largement utilisé à travers le monde. Il propose des fonctionnalités avancées facilitant la programmation, comme la coloration syntaxique, le repérage des erreurs et bien d'autres. De plus, les extensions qui peuvent être ajoutées en font un outil professionnel.

- **Apache**

Apache est un serveur web rapide et sécurisé, qui, en tant que serveur web, est responsable de traiter les requêtes HTTP et d'envoyer une réponse sous la forme d'une page web. Je ne peux pas entrer dans les détails sur ce qu'est un serveur web, car cela sortirait du cadre de la tâche qui nous a été confiée. Cependant, je vous invite fortement à en comprendre le fonctionnement.

- **MYSQL**

MySQL est un système de gestion de base de données fiable, performant, évolutif et facile à utiliser. Un système de gestion de base de données interprète des requêtes données dans un langage de programmation (SQL), applique un algorithme, puis renvoie les informations demandées. Comme tous les outils présentés, il est extrêmement populaire.

- **PHP**

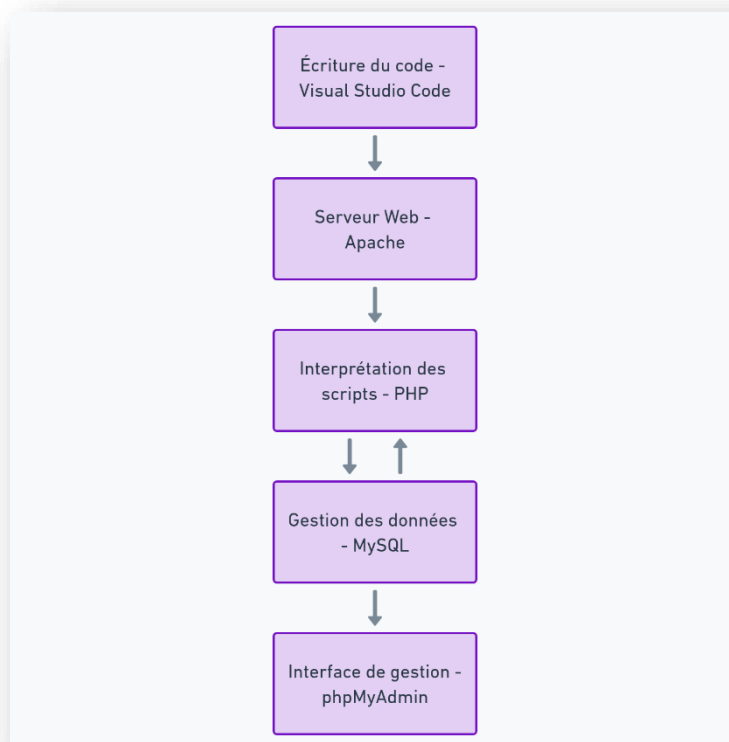
PHP est un choix personnel. Malgré sa cote de popularité qui tend à baisser, il reste un choix très solide pour construire des applications web dynamiques et interactives. De plus, sa documentation détaillée et sa large communauté en font un choix idéal pour le débogage.

- **phpMyAdmin**

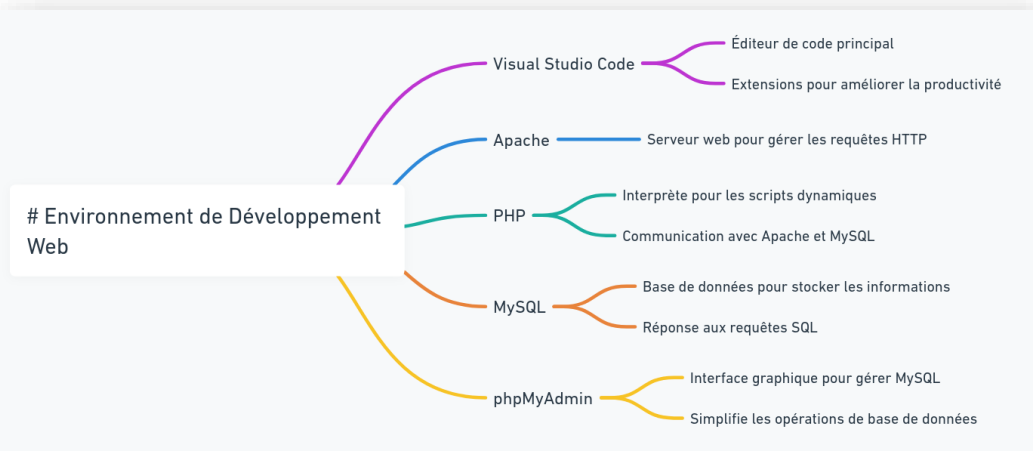
phpMyAdmin est un outil open-source qui offre une interface graphique simplifiée pour gérer les bases de données MySQL. Il permet d'effectuer facilement des opérations comme la création, la modification et la suppression de tables ou de données, tout cela sans avoir à écrire directement des requêtes SQL complexes.

Ces logiciels représentent le parfait kit d'un développeur web. Ils permettront à Raphael de travailler dans un environnement fonctionnel et propice aux travaux. Avec ces outils, il pourra créer des applications web complexes pouvant contenir un front-end et un back-end.

Voici un « diagramme de flux », il représente le schéma de l'architecture logicielle:

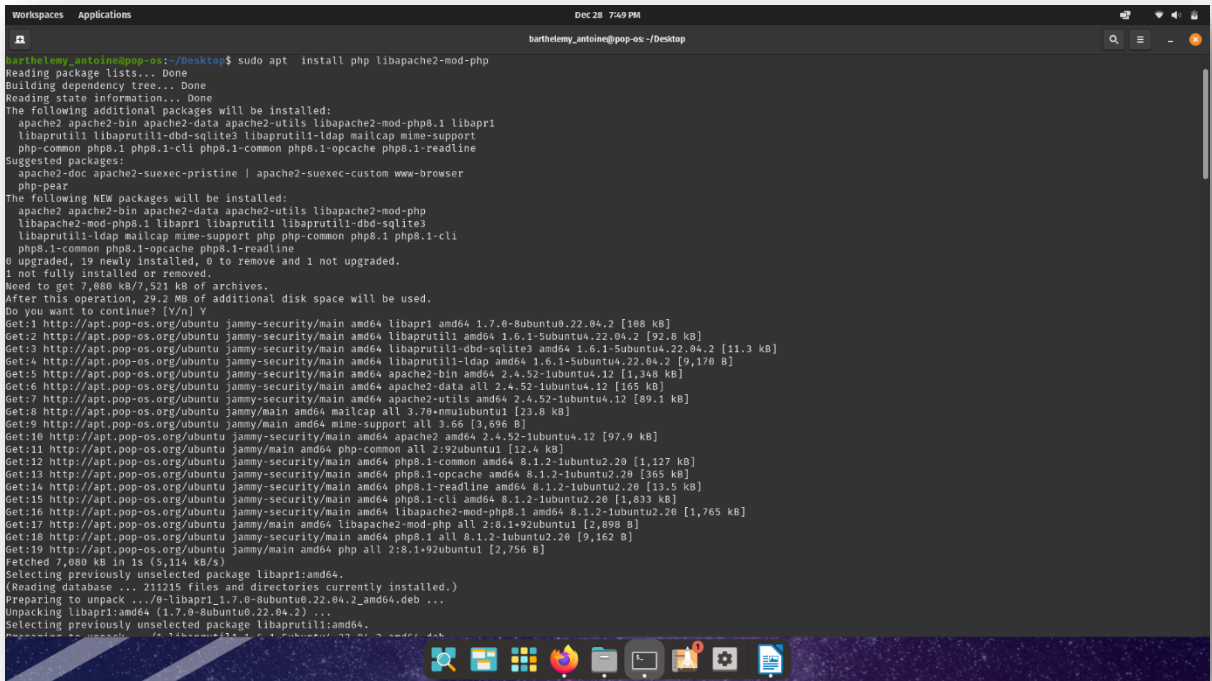


Ce diagramme en arbre illustre la structure hiérarchique des logiciels et leurs rôles dans un environnement de développement web :



Installation des logiciels

Pour installer les logiciels sous POP!_OS plusieurs manière de faire était à ma disposition. J’ai choisit d’utiliser APT (Advanced Package Tool) en ligne de commande. APT est un gestionnaire de packet utilisé sur les systèmes basé sur Debian, il permet d’installer, mettre à jour et supprimer des logiciels. Il est très puissant car il permet de gérer les dépendances entre les logiciels. Je vous invite à vous renseigner à cette adresse: [https://en.wikipedia.org/wiki/APT_\(software\)](https://en.wikipedia.org/wiki/APT_(software))

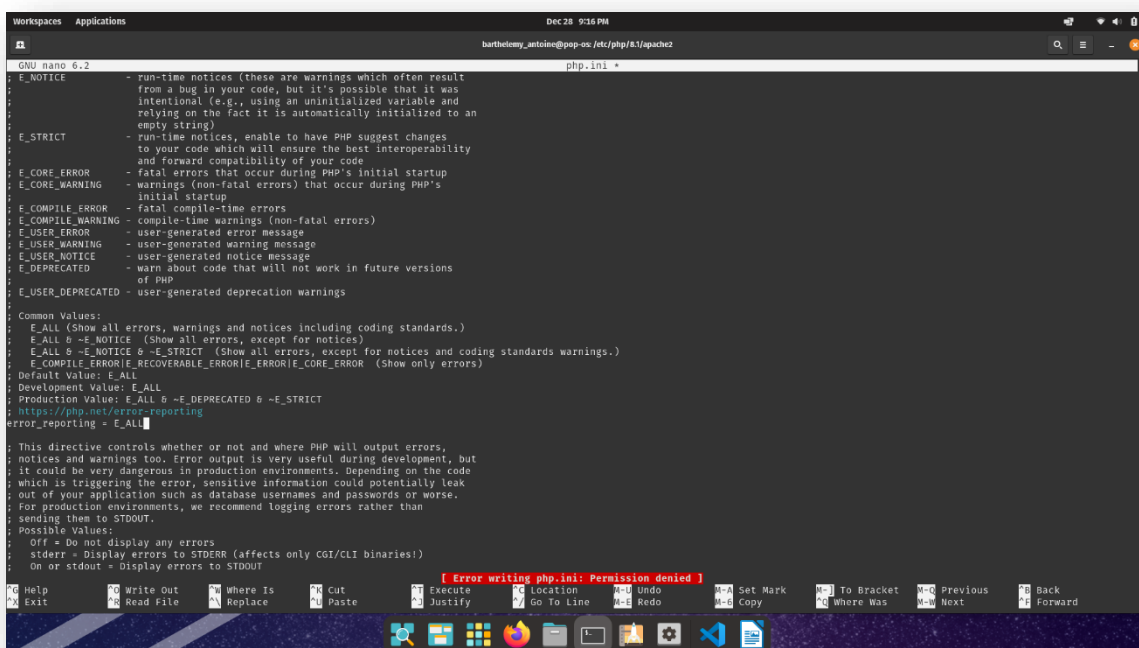


APT propose également une interface graphique (GUI), mais je souhaitais m'exercer à utiliser le terminal. Les autres logiciels ont été installés de la même manière. Cependant, pour certains paquets, il était nécessaire de les inclure dans le dépôt global afin de pouvoir y accéder et les installer.

Configuration des logiciels

- Configuration de PHP/VSCODE

Capture d'écran prise dans le fichier « php.ini »

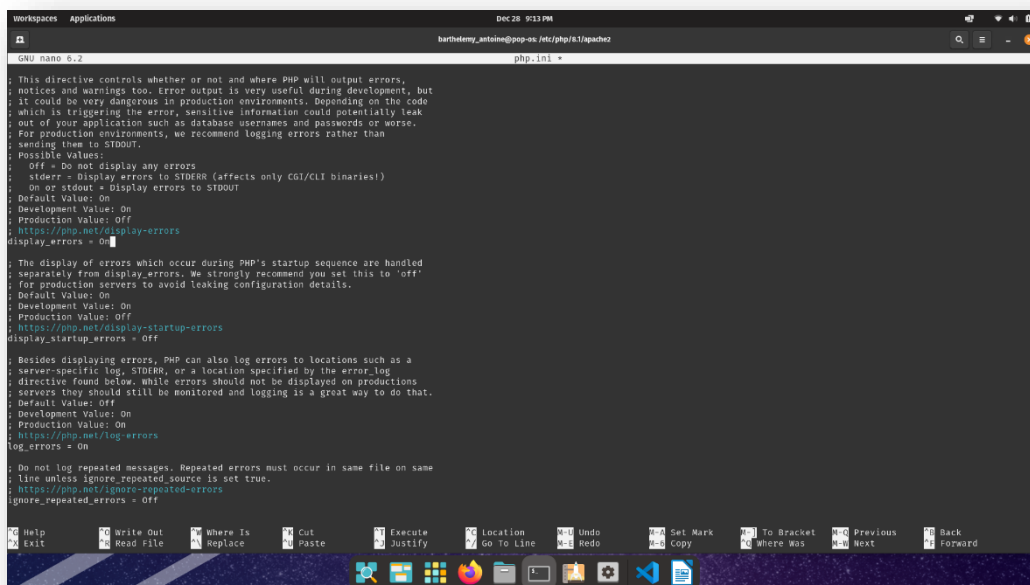


```
GNU nano 6.2 php.ini
; E_NOTICE -> run-time notices (these are warnings which often result
; from a bug in your code, but it's possible that it was
; intentional (e.g., using an uninitialized variable and
; relying on the fact it is automatically initialized to an
; empty string))
; E_STRICT -> run-time notices, enable to have PHP suggest changes
; to your code which will ensure the best interoperability
; and forward compatibility of your code
; E_CORE_ERROR -> fatal errors that occur during PHP's initial startup
; E_CORE_WARNING -> warnings (non-fatal errors) that occur during PHP's
; initial startup
; E_COMPILE_ERROR -> fatal compile-time errors
; E_COMPILE_WARNING -> compile-time warnings (non-fatal errors)
; E_USER_ERROR -> user-generated error message
; E_USER_WARNING -> user-generated warning message
; E_USER_NOTICE -> user-generated notice message
; E_DEPRECATED -> warn about code that will not work in future versions
; of PHP
; E_USER_DEPRECATED -> user-generated deprecation warnings

Common Values:
; E_ALL (Show all errors, warnings and notices including coding standards.)
; E_ALL & ~E_NOTICE (Show all errors, except for notices)
; E_ALL & ~E_NOTICE & ~E_STRICT (Show all errors, except for notices and coding standards warnings.)
; E_COMPILE_ERROR|E_RECOVERABLE_ERROR|E_ERROR|E_CORE_ERROR (Show only errors)
Default Value: E_ALL
Development Value: E_ALL
Production Value: E_ALL & ~E_DEPRECATED & ~E_STRICT
https://php.net/error-reporting
error_reporting = E_ALL

; This directive controls whether or not and where PHP will output errors,
; notices and warnings too. Error output is very useful during development, but
; it could be very dangerous in production environments. Depending on the code
; which is triggering the error, sensitive information could potentially leak
; out of your application such as database usernames and passwords or worse.
; For production environments, we recommend logging errors rather than
; sending them to STDOUT.
Possible Values:
; Off = Do not display any errors
; stderr = Display errors to STDERR (affects only CGI/CLI binaries!)
; On or stdout = Display errors to STDOUT

[Error writing php.ini: Permission denied]
```



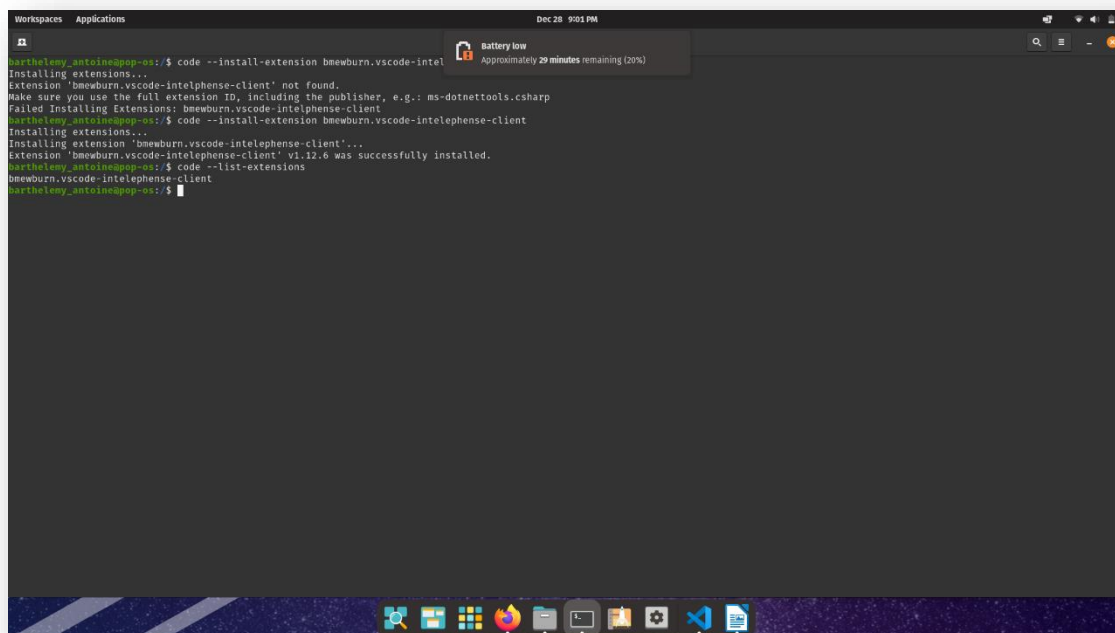
```
GNU nano 6.2 php.ini
; This directive controls whether or not and where PHP will output errors,
; notices and warnings too. Error output is very useful during development, but
; it could be very dangerous in production environments. Depending on the code
; which is triggering the error, sensitive information could potentially leak
; out of your application such as database usernames and passwords or worse.
; For production environments, we recommend logging errors rather than
; sending them to STDOUT.
Possible Values:
; Off = Do not display any errors
; stderr = Display errors to STDERR (affects only CGI/CLI binaries!)
; On or stdout = Display errors to STDOUT
Default Value: On
Development Value: On
Production Value: Off
https://php.net/display-errors
display_errors = On

; The display of errors which occur during PHP's startup sequence are handled
; separately from display_errors. We strongly recommend you set this to 'off'
; for production servers to avoid leaking configuration details.
Default Value: On
Development Value: On
Production Value: Off
https://php.net/display-startup-errors
display_startup_errors = Off

; Besides displaying errors, PHP can also log errors to locations such as a
; server-specific log, STDERR, or a location specified by the error_log
; directive found below. While errors should not be displayed on production
; servers they should still be monitored and logging is a great way to do that.
Default Value: Off
Development Value: On
Production Value: On
https://php.net/log-errors
log_errors = On

; Do not log repeated messages. Repeated errors must occur in same file on same
; line unless ignore_repeated_source is set true.
https://php.net/ignore-repeated-errors
ignore_repeated_errors = Off
```

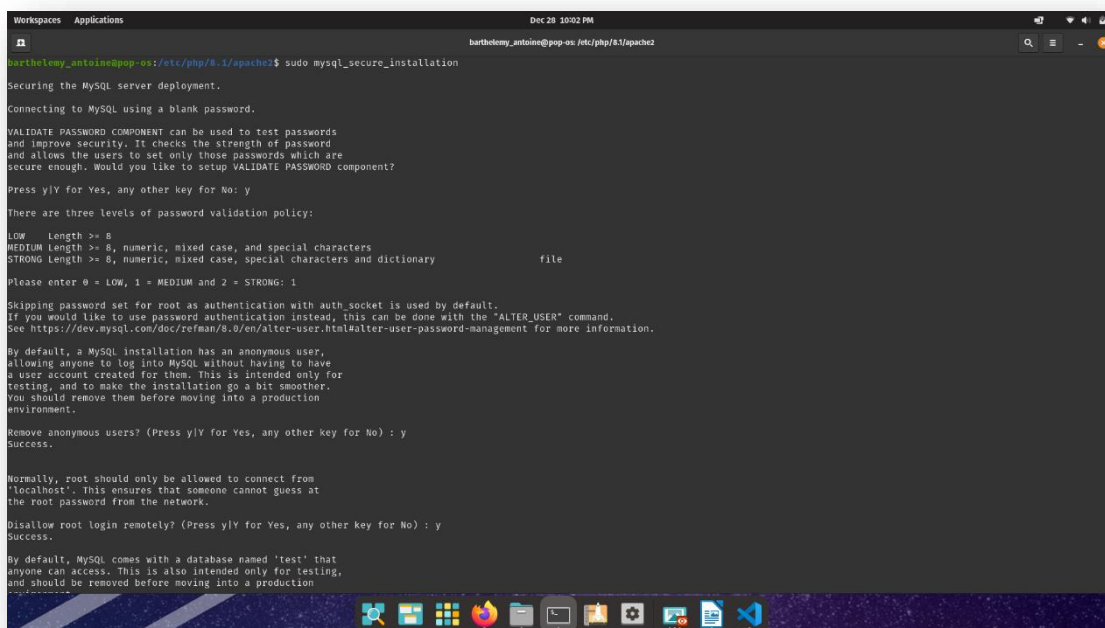
Deux réglages importants doivent être effectués à faire dans le fichier de configuration php. En activant ces deux options, il est possible d'afficher tous les messages d'erreurs ce qui facilite le débogage.



Nous avons également pris soin d'installer une extension pour faciliter le développement en PHP. Cette extension s'avéra utile lorsque Raphael recherchera à être plus productif (coloration syntaxique, autocomplétion ...)

- **Configuration de MySQL**

Cette capture d'écran illustre la sécurisation de MySQL



- Configuration de phpMyAdmin



Ajout d'une couche de sécurité (un mot de passe avant que l'utilisateur accède à la page d'accueil phpMyAdmin pour se connecter)